2024 IEEE 63rd Conference on Decision and Control (CDC)
December 16-19, 2024. MiCo, Milan, Italy

2024 IEEE 63rd Conference on Decision and Control (CDC) | 979-8-3503-1633-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/CDC56724.2024.10886170

Leveraging Control Inputs to Enforce Constraints in Differential Dynamic Programming for Nonlinear Optimization*

Z. Dastan, *Student Member, IEEE*, J. Sensinger, *Senior Member, IEEE*

Abstract— **Differential Dynamic Programming (DDP) has become a popular strategy for optimizing nonlinear dynamic systems due to its algorithmic efficiency and precision in complex control tasks. The goal to integrate inequality constraints into DDP has sparked considerable interest, aiming to extend its utility to more demanding situations with strict operational constraints. This study provides an extension to the conventional control-limited DDP framework, introducing a methodology that leverages control inputs during the backward pass to incorporate inequality constraints. Our methodology enhances the efficiency of DDP and expedites its convergence in a variety of scenarios. We present our method in two variants: the first handles inequality constraints that are functions of both state and control variables, and the second leverages the concept of relative degree from nonlinear control theory to handle inequality constraints that are solely dependent on state variables. Through simulations on an inverted pendulum and a nonholonomic 2D car, we benchmark our approach against established methods such as Constrained DDP (CDDP) and primal–dual interior-point DDP (IPDDP). The results showcase our method's superior convergence rate and trajectory efficiency, particularly highlighting the efficacy of employing control inputs for constraint enforcement.**

Keywords—Differential dynamic programming (DDP), inequality constraints, control optimization, nonlinear system optimization

I. INTRODUCTION

Differential Dynamic Programming (DDP) has become a cornerstone in nonlinear system optimization, particularly in robotics where precise motion control is paramount [1], [2]. DDP distinguishes itself through an iterative approach based on Bellman's principle, optimizing input trajectories without direct parameterization of state trajectories [3]. Its algorithmic efficiency and ability to handle complex dynamics with high accuracy make DDP particularly suitable for real-time applications and scenarios requiring rapid decision-making. Additionally, the adaptability of DDP to various control and dynamics models extends its applicability beyond traditional robotics, into areas such as autonomous vehicles [4], [5], aerospace [6], and energy systems [7], demonstrating its broad utility in tackling a wide range of optimization problems. However, incorporating inequality constraints directly into the DDP framework poses a significant challenge, particularly when dealing with complex scenarios that require stringent adherence to safety and operational limits. This limitation necessitates approaches to modify or extend DDP to extend its effectiveness and applicability across a diverse range of constrained optimization problems [8].

A. Recent progress

The evolution of handling inequality constraints within the DDP framework has seen the development of various methods, each with its unique approach to embedding these constraints into the optimization process [9], [10]. Penalty methods [11], for example, add significant costs for deviating from constraints to the objective function, effectively deterring solutions that violate these limits. This approach simplifies the computational process by treating a constrained optimization problem as an unconstrained one. However, this method faces challenges such as potential numerical instability (ill-conditioning) [12] and the difficulty of selecting appropriate penalty parameters.

On the other hand, barrier functions [13], [14] are employed to create nearly insurmountable 'barriers' at the edges of feasible regions, ensuring that the optimization process remains within the bounds of constraint adherence. While barrier functions are adept at guiding the solution to stay within the feasible space, they also require meticulous parameter tuning to avoid ineffective constraint enforcement or excessively slow convergence [15]. Interior point methods [13] offer another technique for handling constraints, maintaining the search within the feasible region by utilizing barrier functions . These methods progress from the interior towards the boundary of the feasible region and require a mechanism to effectively vary the perturbation parameter during the optimization process. Additionally, they typically do not benefit from warm starts, which can be a limitation depending on the application.

Active set methods [15] present a different approach by embedding the constraints directly into the optimization process, allowing for the exploitation of warm starts and potentially reducing the number of iterations required. These methods necessitate that the constraints be explicitly dependent on the control variables, which may limit their application in scenarios where constraints are dependent on state variables [1], [16].

*Research supported by Natural Sciences and Engineering Research Council of Canada (NSERC).

Z. Dastan is with the Department of Electrical Engineering, University of New Brunswick, Fredericton, NB, Canada (e-mail: zahed.dastan@unb.ca). J. Sensinger is also with the Department of Electrical Engineering, University of New Brunswick, Fredericton, NB, Canada (e-mail: j.sensinger@unb.ca)

B. Paper contribution

The foundational work in [1] advances DDP by incorporating box inequality constraints on control inputs through active set methods. This methodology integrates control constraints into the optimization framework without compromising the original DDP algorithm's convergence properties, critical for real-time applications demanding prompt decision-making. Its efficiency is evident in handling multiple smaller Quadratic Programming (QP) problems rather than a singular, larger one, markedly lowering computational complexity. Additionally, its capability to reach the solution in a single iteration when the initial point and the optimum share the same active set underscores its computational effectiveness.

Although the work in [1] can efficiently handle inequality *control* constraints, it does not extend the method to constraints involving *state*, instead relying on the fact that due to the feasibility property of indirect methods, inequality constraints on the state can be handled when enforcing regularity conditions. More recent studies, however, have cited the lack of box-bounded constraints of states as a limitation [16]. Our work accordingly introduces two novel extensions to the framework established in [1] to extend box-bounded constraints to state:

I. We propose a formulation that incorporates inequality constraints involving both control and state variables.

II. We devise a variant specifically for inequality constraints that are functions of state variables alone.

By employing these extensions, we use the robust features of the method outlined in [1], broadening the applicability to a wider spectrum of constrained optimization scenarios. We showcase the efficacy of the proposed methods through several simulations, demonstrating their practical utility and advantages in a variety of settings.

II. Overview of Differential Dynamic Programming

In this section, we briefly recall the derivation and implementation of unconstrained DDP. More details can be found in [2].

Consider a system characterized by its continuous-time behavior

$$\dot{x} = f(x, u), \tag{1}$$

where $x \in \mathbb{R}^n$ denotes the state of the system, $u \in \mathbb{R}^m$ is the control input and $f: \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ represents the system dynamics. For analytical and computational tractability in control applications, this continuous-time system can be approximated by its discrete-time equivalent

$$x_{i+1} = f(x_i, u_i), \tag{2}$$

where $x_i \in \mathbb{R}^n$ represents the state of the system at the discrete time step i, and $u_i \in \mathbb{R}^m$ denotes the corresponding control input.

The optimal control problem is defined by the cost functional

$$J(X, U) = \Phi(x_N) + \sum_{i=0}^{N-1} L(x_i, u_i), \tag{3}$$

where $U \coloneqq \{u_0^T, \dots, u_{N-1}^T\}$, $X \coloneqq \{x_0^T, \dots x_N^T\}$, $L(x_i, u_i)$ represents the running cost at each time step i for the state x_i and control u_i, and $\Phi(x_N)$ is the terminal cost at the final state x_N. The objective is to determine the optimal control sequence U that minimizes the cost functional J subject to the system dynamics given by (2) with a given initial state x_0.

The Value function, $V(x)$ quantifies the minimal cost-to-go from any given state x_i to the final state under an optimal control policy. Formally, the Value function at time step i in a finite-horizon optimization framework is defined as

$$V_i(x_i) = \min_{u_i}\{L(x_i, u_i) + V_{i+1}(f(x_i, u_i))\}, \tag{4}$$

with the terminal condition given by $V_N(x_N) = \Phi(x_N)$. The objective of DDP is to find local optimal solution for (3) by expanding both sides of (4) about a nominal trajectory $\bar{X}, \bar{U}$.

Let $Q_i(\delta x_i, \delta u_i)$ denote the change in the argument of the right-hand side of (4)

$$\begin{aligned} Q_i(\delta x_i, \delta u_i) = {} & L(x_i + \delta x_i, u_i + \delta u_i) \\ & + V_{i+1}(f(x_i + \delta x_i, u_i + \delta u_i)). \end{aligned} \tag{5}$$

Note that Q encapsulates the immediate cost associated with small deviations, δx and δu, in the state and control vectors, respectively. The second-order expansion of Q is

$$\begin{aligned} Q_i(\delta x_i, \delta u_i) \approx {} & Q_i + Q_{x,i}^T \delta x_i + Q_{u,i}^T \delta u_i + \frac{1}{2}\delta x_i^T Q_{xx,i} \delta x_i \\ & + \delta x_i^T Q_{xu,i} \delta u_i + \frac{1}{2}\delta u_i^T Q_{uu,i} \delta u_i, \end{aligned} \tag{6}$$

with

$$\begin{aligned} Q_{xx,i} &= L_{xx,i} + f_x^T V_{xx,i+1} f_x + V_{x,i+1} \cdot f_{xx}, \\ Q_{xu,i} &= L_{xu,i} + f_x^T V_{xx,i+1} f_u + V_{x,i+1} \cdot f_{xu}, \\ Q_{uu,i} &= L_{uu,i} + f_u^T V_{xx,i+1} f_u + V_{x,i+1} \cdot f_{uu}, \\ Q_{u,i} &= L_{u,i} + f_u^T V_{x,i+1}, \\ Q_{x,i} &= L_{x,i} + f_x^T V_{x,i+1}. \end{aligned} \tag{7}$$

Now the optimal control perturbation δu_i^* for some δx_i is obtained by minimizing (6)

$$\delta u_i^* = k_i + K_i \delta x_i, \tag{8}$$

where $k_i = -Q_{uu,i}^{-1} Q_{u,i}$ and $K_i = -Q_{uu,i}^{-1} Q_{ux,i}$. By substituting (8) back into the quadratic model of the Value function V_i, we get

$$\begin{aligned} V_{x,i} &= Q_{x,i} - Q_{xu,i} Q_{uu,i}^{-1} Q_{u,i}, \\ V_{xx,i} &= Q_{xx,i} - Q_{xu,i} Q_{uu,i}^{-1} Q_{ux,i}. \end{aligned} \tag{9}$$

The backward pass in DDP starts by setting the Value function to the terminal cost $V(x_N) = \Phi(x_N)$, and proceeds to calculate its derivatives [1]. It then iteratively computes the

optimal control perturbations and the corresponding updates to the Value function, as encapsulated in equations (8) and (9). Upon completion of the backward pass, the forward pass applies the derived control sequence to update the system states. The iterative process continues until convergence criteria—such as a sufficiently small change in the control sequence or the cost-to-go—are met, indicating that an optimal or near-optimal control policy has been found.

III. Integrating Constraints into DDP

This section delineates the theoretical underpinnings of the proposed extensions to DDP for handling inequality constraints. Our methodology is divided into two primary extensions, each designed to address specific types of inequality constraints and broaden the applicability of DDP in constrained optimization scenarios.

A. Integrating inequality constraints involving control and input variables

Our first extension is motivated by integrating inequality constraints of the form

$$h(x,u) \geq 0, \tag{10}$$

where $h(x,u) \in \mathbb{R}^P$, into the DDP framework. These constraints are commonly encountered across a variety of applications such as robotics [17], and vehicle control [18], to name a few.

In contrast to [1], who assumes a fixed boundary of u along the entire trajectory, our method introduces an iterative adjustment of control input boundaries, a process that is confined to the backward pass phase of DDP. This adjustment is based on the current nominal trajectory, x_i, for $i = 0, \dots, N$, succinctly formulated as

$$h(x_i,u) \geq 0 \; for \; i = 0, \dots, N. \tag{11}$$

Such constraints inform the dynamic establishment of control input boundaries

$$u_i \in \left[u_i^{min}(x_i), u_i^{max}(x_i)\right], \quad i = 0, \dots, N. \tag{12}$$

Given these dynamically adjusted boundaries, we then articulate the optimization problem at each timestep as follows

$$\begin{aligned} &\min_{\delta u_i} Q\,(\delta x_i, \delta u_i) \\ &\text{subject to} \quad u_i^{min}(x_i) \leq u_i + \delta u_i \leq u_i^{max}(x_i). \end{aligned} \tag{13}$$

For the sake of compactness and clarity in later discussions, we will omit the explicit dependence on x from $u_i^{min}(x)$ and $u_i^{max}(x)$.

We can then address this optimization problem by employing the method proposed in [1] with the extension of having a state-informed varying box constraint of u rather than the conventional non-state informed constant boundary of u. This approach modifies the traditional DDP algorithm to directly incorporate control input constraints into the optimization procedure. Utilizing Tassa et al. method offers distinct advantages. Notably, one of its key features is the capability to converge to the optimal solution within a single iteration if the initial point shares the same active constraint set with the optimum. This efficiency is a direct consequence of employing the Projected-Newton class of algorithms, which are adept at handling simple constraints through the projection operator. These algorithms facilitate a dynamic adjustment process where the search point is continuously clamped to respect multiple constraints, thereby enabling a highly efficient and targeted approach to constraint management within the DDP framework.

Note that our approach streamlines the process of incorporating inequality constraints by specifically satisfying the constraints through the control inputs. The constraint (11) is used to determine the bounds for the control input u, rather than simultaneously considering the state and control spaces. This approach significantly reduces complexity as it circumvents the need to resolve the constraints in the higher-dimensional joint space of (x,u) [16]. In formal terms, assuming that the system's dynamics are given by (2), which can be thought of as a vector field in the state space x parametrized by u, our approach involves selecting control directions so that the resulting trajectory inherently satisfies (11).

To visualize the practical implementation of this methodology, consider the following process. Assume that from some initial condition x_0, and initial control input u_0, a reference trajectory has been generated during the forward pass, depicted by dashed line in Fig. 1. During the backward pass, starting from x_N, for each point x_i along this trajectory, we compute u_i^{min} and u_i^{max}, resulting in a set of vectors given by $f(x_i,u_i), u_i \in \left[u_i^{min}, u_i^{max}\right]$. The algorithm then selects the optimal δu_i to minimize Q, ensuring compliance with the constraints. In the subsequent forward pass, the reference trajectory is updated according to these optimized control inputs, reflected in Fig. 2 with a solid line.

It is worth mentioning that as long as the adjustments to u are modest and $h(x,u)$ is smooth, the updated trajectory will continue to fulfill the inequality constraint

$$h(x_i + \delta x_i, u_i + \delta u_i) \geq 0 \; for \; i = 0, \dots, N. \tag{14}$$

By focusing on adjusting u within its allowable range to ensure compliance with the constraints, we bypass the intricacies that arise from the interdependencies between x and u. This targeted strategy not only simplifies the computational procedure but also aligns with the intuitive understanding that

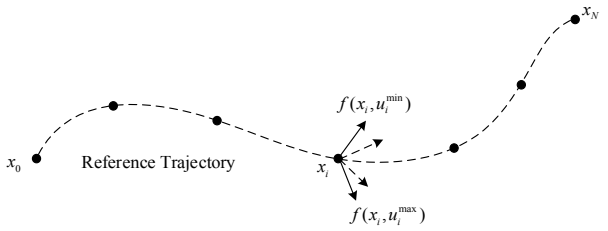


Fig. 1. Calculating u_i^{min} and u_i^{max} during backward pass at each point x_i of the reference trajectory, which yields range of vectors $f(x_i,u_i), u_i \in \left[u_i^{min}, u_i^{max}\right]$. Modest adjustments along these vectors ensure (11) is still satisfied.

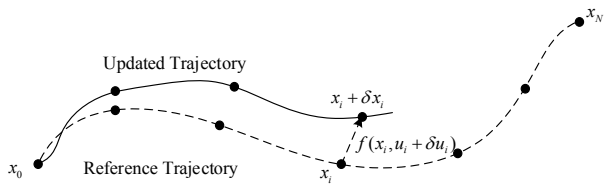


Fig. 2. Updating the reference trajectory during the forward pass.

control inputs are often the direct levers for ensuring constraint satisfaction.

B. Constraints without explicit control input

In this subsection, we broaden the scope of our method to accommodate constraint solely dependent on the state, denoted as

$$h(x) \geq 0, \tag{15}$$

where $h(x) \in \mathbb{R}^P$, and it does not explicitly involve the control input u. To integrate such constraints within our framework, we turn to the concept of relative degree from nonlinear control theory. Assume (15) exhibits a relative degree one with respect to one control input. This assumption implies that the time derivative of $h(x)$ denoted as $\dot{h}(x)$ and given by

$$\dot{h}(x) = \frac{\partial h}{\partial x}\dot{x}, \tag{16}$$

transforms with the system dynamics (1), into

$$\dot{h}(x) = \frac{\partial h}{\partial x} f(x, u), \tag{17}$$

which explicitly depends on u. Now to enforce (15) within the context of our DDP framework, we construct the differential equation

$$\dot{h}(x) + \alpha h(x) \geq 0, \tag{18}$$

where $\alpha > 0$ is a constant that can be determined through iterations, ensuring that the constraint is not violated. Substituting $\dot{h}(x)$ from (17) yields

$$\frac{\partial h}{\partial x} f(x, u) + \alpha h(x) \geq 0. \tag{19}$$

This formulation introduces a condition that dynamically adjusts the system's behavior to maintain compliance with the constraint. Specifically, if $h(x)$ were at risk of becoming negative, the positive contribution from $\alpha h(x)$ encourages a corrective increase in $\dot{h}(x)$, directing $h(x)$ back towards positive values. This approach ensures the system's states remain within the permissible region by dynamically enforcing the constraints without the need for explicit control input in the constraint equation.

With the reformulated constraint now implicitly involving the control input u, we can directly apply the methodology developed in the previous subsection to our current problem. By translating the state-dependent constraints into a form that includes the control input u indirectly, we empower our DDP framework to handle a broader range of constraints.

IV. Simulation Results

In this section, to assess the performance of our proposed DDP algorithm, we conducted a series of simulations across two distinct scenarios: inverted pendulum [13], and a nonholonomic 2D car [1]. Our objective was to benchmark the proposed DDP algorithm against two well-established methods: the Constrained DDP (CDDP) [16] and the primal–dual interior-point DDP (IPDDP) [13]. For all simulations, the initial control input u_0 is selected by generating small random values, providing a diverse starting point for the optimization process. These simulations are designed to test the algorithm's capability in handling different dynamic systems and control challenges. To effectively evaluate and compare these algorithms, while recognizing the importance of iteration time, we selected the number of iterations as our primary performance metric [13]. This choice is rooted in providing a hardware-independent assessment of each algorithm.

A. Inverted pendulum

In our first simulation, we evaluate the effectiveness of our proposed DDP method in controlling an inverted pendulum model, in comparison with the CDDP and the IPDDP methods. The system is characterized by the states $X = [\phi, \omega]^T$, and following discrete-time dynamic equations

$$\begin{aligned} \phi_{i+1} &= \phi_i + h\omega_i, \\ \omega_{i+1} &= \omega_i + hsin(\phi_i) + hu_i, \end{aligned} \tag{20}$$

where ϕ is the angle, ω is the angular velocity, and u denotes the control input. The task is to drive the pendulum from an initial state $X_0 = [-\pi, 0]^T$ to $X_{final} = [0,0]^T$, over a control horizon of $N = 500$, and $h = 0.05$. We incorporate a state-dependent constraint to maintain the angular velocity below a certain threshold

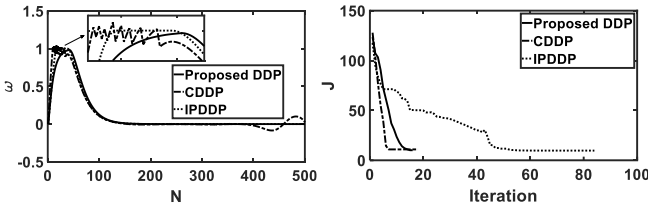
$$\omega < 1. \tag{21}$$

Given that (21) has a relative degree of one, we employ (19) with $\alpha = 0.1$ for our proposed DDP algorithm. The running cost and the terminal cost used for the simulations across all three algorithms are defined as

$$L(X, u) = 0.025(\phi^2 + \omega^2 + u^2), \tag{22}$$

$$\Phi(X) = 5(\phi^2 + \omega^2). \tag{23}$$

Fig. 3a illustrates the trajectories of the angular velocity, ω, as a function of the control horizon N. It is observed that while all three algorithms successfully drive ω close to 0, the CDDP algorithm's trajectory shows fluctuations near the constraint boundary, which is not desirable. Fig. 3b presents the evolution of the cost function J with respect to the number of iterations. Here, the proposed algorithm and CDDP demonstrate rapid convergence within fewer than 20 iterations, specifically 8 for CDDP and 16 for the proposed method. In contrast, IPDDP requires nearly 60 iterations to stabilize.



(a) (b)

Fig. 3. Comparative analysis of DDP algorithms on the inverted pendulum model. (a) Angular velocity (ω) trajectories. (b) Cost function (J) convergence.

Notably, our proposed algorithm achieves a final cost of 10.11 with an approximate ω_{final} of 0, outperforming CDDP, which concludes with a final cost of 10.92 and an ω_{final} of approximately 0.02. The IPDDP method attains the best final cost of 9.72, albeit with a higher iteration count, yet still arriving at an ω_{final} close to 0. These results affirm the robustness of our proposed method in efficiently reaching the target state while also adhering to the state-dependent constraint imposed on the system.

B. 2D Car

In the subsequent example, we turn our attention to the application of our DDP algorithm on a 2D car navigation scenario. The car is modeled with a state vector $X = [x, y, \theta, v]^T$, delineating the car's planar position (x, y), angle θ, and front wheel velocity v. The control inputs are represented by $u = [\omega, a]^T$, which consist of the steering angle ω and acceleration a. The dynamics of the car are given by

$$\begin{aligned} x_{i+1} &= x_i + b\cos\theta_i, \\ y_{i+1} &= y_i + b\sin\theta_i, \\ \theta_{i+1} &= \theta_i + sin^{-1}\left(\frac{h}{d}sin(\omega_i)\right), \\ v_{i+1} &= v_i + ha_i, \end{aligned} \tag{24}$$

where $b = d + hv\cos(\omega) - \sqrt{d^2 - h^2v^2\sin^2(\omega)}$. The constants are set as $d = 2.0$, representing the distance between the rear and front axles, and $h = 0.03$, which represents the time step.

1) Comparison with CDDP

The navigation task involves maneuvering the car from the initial state $X_0 = [1,1,3\pi/2\,,0]^T$ to the destination $X_{final} = [0,0,0,0]^T$, circumventing a circular obstacle centered at $[-2,-3]$ with a radius of 1 defined by

$$(x+2)^2 + (y+3)^2 \geq 1. \tag{25}$$

Note that (25) has a relative degree of one; therefore, we utilize (19) with $\alpha = 0.1$ for the proposed DDP. The horizon is set to $N = 500$, with steering angle constraint of $-\pi/2 \leq \omega \leq \pi/2$ and acceleration constraint of $-8 \leq a \leq 8$ for both algorithms. The cost function employed for both CDDP and the proposed DDP is composed of a running cost

$$\begin{aligned} L(X,u) = 10^{-3}r(x,0.1) + 10^{-3}r(y,0.1) \\ +10^{-2}(\omega^2 + 0.01a^2), \end{aligned} \tag{26}$$

and a terminal cost

$$\begin{aligned} \Phi(X) = 0.1r(x,0.01) + 0.1r(y,0.01) + r(\theta,0.01) \\ +0.3r(v,1), \end{aligned} \tag{27}$$

where $r(x,p) = \sqrt{x^2+p^2} - p$ represents the Huber-type loss function [1].

Fig. 4a illustrates the trajectories generated by both the proposed DDP and the CDDP algorithms as they navigate towards the destination while avoiding the circular obstacle. The effectiveness of both methods is demonstrated by their successful avoidance of the obstacle and their convergence towards the final desired state. Specifically, the final state achieved by the CDDP algorithm is $X = [0.007,0.003,0.002,-0.228]^T$, while the proposed DDP method reaches a final state of $X = [0,0.006,0.002,0]^T$. Furthermore, the cost function J for both proposed and CDDP algorithms are depicted in Fig. 4b, offering a quantitative measure of their performance. Notably, the proposed DDP algorithm exhibits a faster convergence, requiring only 25 iterations to reach a steady value, as opposed to the 45 iterations needed by the CDDP method. Despite this, the CDDP algorithm demonstrates a slight advantage in minimizing the total cost, achieving an optimal cost of 1.086 compared to 1.067 for the proposed DDP method.

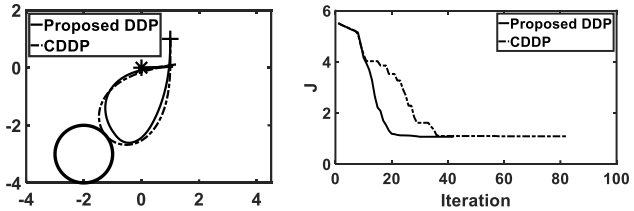
2) Comparison with IPDDP

In this subsection, we present a comparative analysis of our proposed DDP algorithm with the IPDDP method. The evaluation is carried out in a scenario involving navigation through a field with three circular obstacles. The navigation task commences from the initial state $X_0 = [0,0,0,0]^T$ and aims to reach the destination $X_{final} = [3,3,\pi/2\,,0]^T$. The environment comprises three circular obstacles located at $[1,1]$, $[2.5,2.5]$, $[1,2.5]$, each with a radius of 0.5. Due to the relative degree of one presented by the obstacles, we apply (19) with distinct α parameters— $\alpha_1 = 0.2, \alpha_2 = 0.01$, and $\alpha_3 = 0.02$—for each corresponding constraint.

The horizon is set at $N = 200$, with the steering angle ω constrained within $-\pi/2 \leq \omega \leq \pi/2$. The running cost for both the proposed DDP and IPDDP algorithms is defined as follows

$$\begin{aligned} L(X,u) = 10^{-4}r(x,0.1) + 10^{-3}r(y,0.1) \\ +10^{-4}(\omega^2 + 0.01a^2). \end{aligned} \tag{28}$$

The terminal cost is given by



(a) (b)

Fig. 4. Performance comparison in 2D car navigation for the proposed and CDDP algorithms. (a) Trajectory paths of the proposed and CDDP algorithms. (b) Cost function (J) convergence.

$$\Phi(X) = 0.06\big(0.1r(x, 0.01) + 0.1r(y, 0.01)\big) + 0.06r(\theta, 0.01) + 0.018r(v, 1). \quad (29)$$

This cost function has been selected to accommodate the changed obstacle structure, ensuring that both the proposed DDP and IPDDP algorithms converge efficiently.

Fig. 5a displays the paths computed by the proposed DDP and IPDDP algorithms. While both methods adeptly navigate around the obstacles and converge towards the target, it is noteworthy that the proposed DDP algorithm yields a more direct path compared to the circuitous route taken by the IPDDP. The final state reached by the IPDDP is $X = [3,3.01,1.57,-0.03]^T$, while the proposed method achieves $X = [3.25,3.15,1.57,0.41]^T$. These results suggest that the IPDDP algorithm has a slight edge in reaching the precise X_{final}. Nonetheless, when examining the cost function J, depicted in Fig. 5b, it is observed that the proposed algorithm converges significantly faster, in only 22 iterations, compared to 160 iterations for the IPDDP. Though the IPDDP achieves a slightly lower final total cost of 0.0615 compared to the 0.0761 of the proposed method.

V. Conclusion

This paper contributes to the field of nonlinear system optimization by exploring the use of control inputs to directly enforce inequality constraints within DDP. We have introduced two variants of this extended DDP framework, targeting constraints associated with both state and control variables, and those specific to state variables alone. Through simulations on an inverted pendulum and a nonholonomic 2D car, our method was compared against established techniques such as CDDP and IPDDP, demonstrating its feasibility and potential advantages in certain aspects of convergence rate and trajectory efficiency.

Our method could be further improved regarding the calculation of boundaries on control inputs, which, in its current form, contributes to longer iteration times. We believe that this aspect could benefit from the development and incorporation of more efficient algorithms, thereby enhancing the overall performance of our approach.

While our findings suggest a promising direction for incorporating constraints into DDP, it is important to recognize the limitations and scope for further research. The slight compromise on cost efficiency compared to IPDDP, for example, underscores the need for ongoing refinement and exploration of our method. Additionally, an essential future direction for our research is the consideration of constraints with a relative degree greater than one. Upcoming work will concentrate on enhancing the computational efficiency of calculating control boundaries and examining the scalability of our methodology to systems of increased complexity.

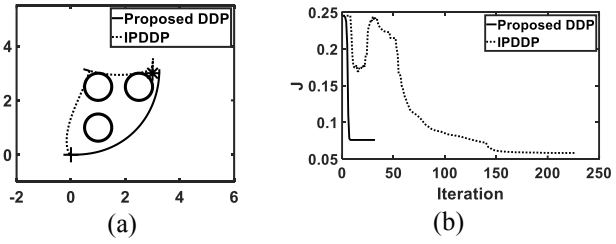


Fig. 5. Performance comparison in 2D car navigation for the proposed and IPDDP algorithms. (a) Trajectory paths of the proposed and IPDDP algorithms. (b) Cost function (J) convergence.

VI. Acknowledgment

I would like to acknowledge the assistance of ChatGPT in reviewing and refining the grammar of this paper.

References

[1] Y. Tassa, N. Mansard, and E. Todorov, "Control-limited differential dynamic programming," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2014, pp. 1168–1175.

[2] D. Mayne, "A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems," *Int. J. Control*, vol. 3, no. 1, pp. 85–95, 1966.

[3] R. B. Vinter and R. B. Vinter, *Optimal control*, vol. 2, no. 1. Springer, 2010.

[4] C. Mastalli *et al.*, "A feasibility-driven approach to control-limited DDP," *Auton. Robots*, vol. 46, no. 8, pp. 985–1005, 2022.

[5] N. Wu, W. Huang, Z. Song, X. Wu, Q. Zhang, and S. Yao, "Adaptive dynamic preview control for autonomous vehicle trajectory following with DDP based path planner," in *2015 IEEE Intelligent Vehicles Symposium (IV)*, IEEE, 2015, pp. 1012–1017.

[6] M. D. Houghton, A. B. Oshin, M. J. Acheson, E. A. Theodorou, and I. M. Gregory, "Path planning: Differential dynamic programming and model predictive path integral control on VTOL aircraft," in *AIAA SCITECH 2022 Forum*, 2022, p. 624.

[7] B. Jeddi, Y. Mishra, and G. Ledwich, "Differential dynamic programming based home energy management scheduler," *IEEE Trans. Sustain. Energy*, vol. 11, no. 3, pp. 1427–1437, 2019.

[8] F. Fioretto, E. Pontelli, and W. Yeoh, "Distributed constraint optimization problems and applications: A survey," *J. Artif. Intell. Res.*, vol. 61, pp. 623–698, 2018.

[9] H. Almubarak, K. Stachowicz, N. Sadegh, and E. A. Theodorou, "Safety embedded differential dynamic programming using discrete barrier states," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2755–2762, 2022.

[10] M. Cho, Y. Lee, and K.-S. Kim, "Model predictive control of autonomous vehicles with integrated barriers using occupancy grid maps," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 2006–2013, 2023.

[11] M. B. Oguntola and R. J. Lorentzen, "Ensemble-based constrained optimization using an exterior penalty method," *J. Pet. Sci. Eng.*, vol. 207, p. 109165, 2021.

[12] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, "Feedback mpc for torque-controlled legged robots," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2019, pp. 4730–4737.

[13] A. Pavlov, I. Shames, and C. Manzie, "Interior point differential dynamic programming," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 6, pp. 2720–2727, 2021.

[14] B. Plancher, Z. Manchester, and S. Kuindersma, "Constrained unscented dynamic programming," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 5674–5680.

[15] J. Breeden and D. Panagou, "Robust control barrier functions under high relative degree and input constraints for satellite trajectories," *Automatica*, vol. 155, p. 111109, 2023.

[16] Z. Xie, C. K. Liu, and K. Hauser, "Differential dynamic programming with nonlinear constraints," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2017, pp. 695–702.

[17] J. W. Grizzle, C. Chevallereau, A. D. Ames, and R. W. Sinnet, "3D bipedal robotic walking: models, feedback control, and open problems," *IFAC Proc. Vol.*, vol. 43, no. 14, pp. 505–532, 2010.

[18] T. Yoshioka, T. Adachi, T. Butsuen, H. Okazaki, and H. Mochizuki, "Application of sliding-mode theory to direct yaw-moment control," *JSAE Rev.*, vol. 20, no. 4, pp. 523–529, 1999.